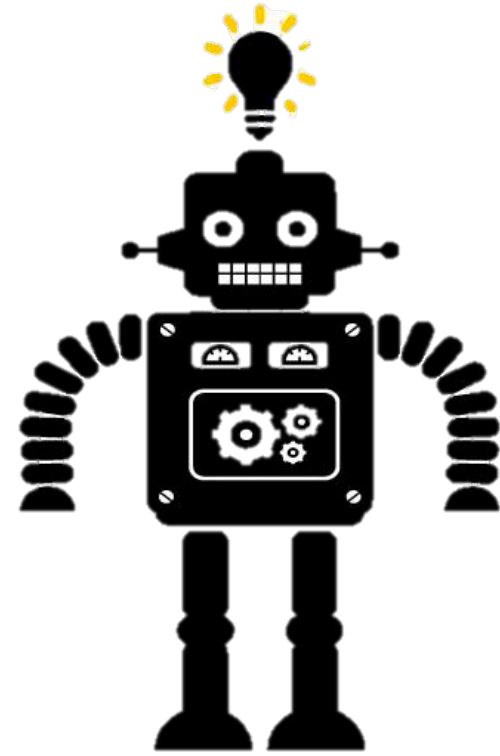


Intro to Deep Learning

Introduction to Python Programming Part 1

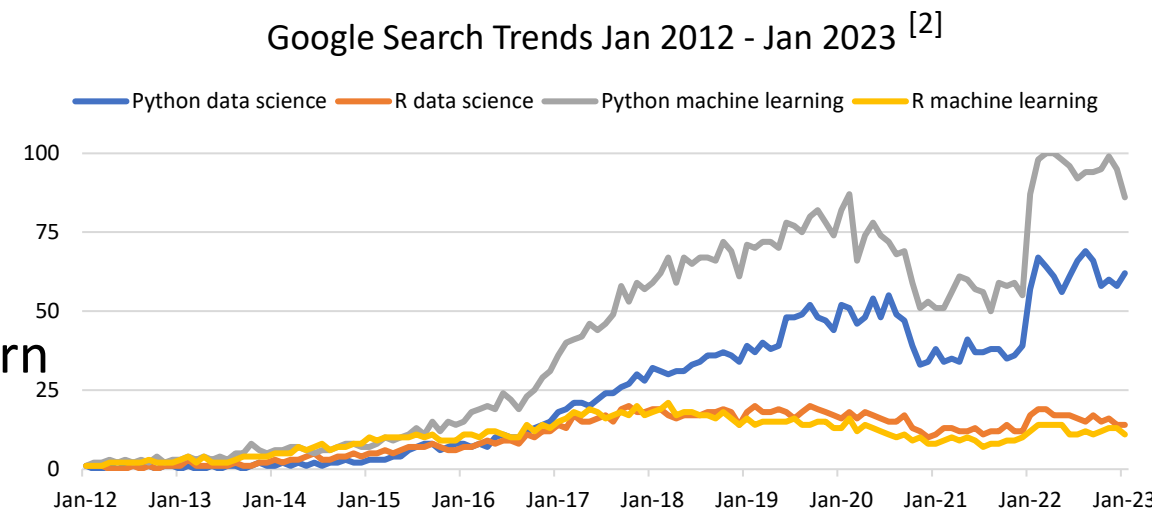


In this lecture

- What is Python programming?
- Resources
- Python IDEs and Editors
 - PyCharm
 - Spyder
 - Google Colab
- Expressions (print, data types, basic calculations)
- Variables (creating them, rules to consider)
- Simple Functions (type casting, calculations, modules, etc.)
- Conditions (comparisons, logic operators, conditional statements)
- Iterations (while, for, break, continue)
- Examples

Python is a programming language that lets you work quickly and integrate systems more effectively

- Python^[1]:
 - Interpreted
 - High-level
 - General-purpose programming language
- It supports:
 - Procedural, object-oriented, and functional programming.
- Python's large standard library
 - Graphical user interfaces
 - Machine learning
 - Image processing and much more.
- Python is commonly used in AI
 - Libraries like TensorFlow, Keras and Scikit-learn

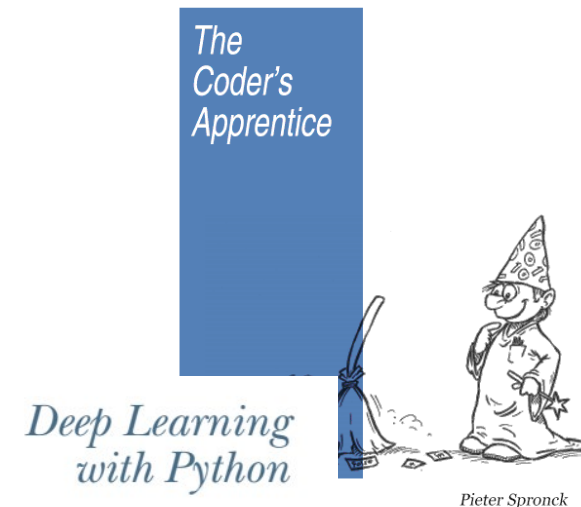
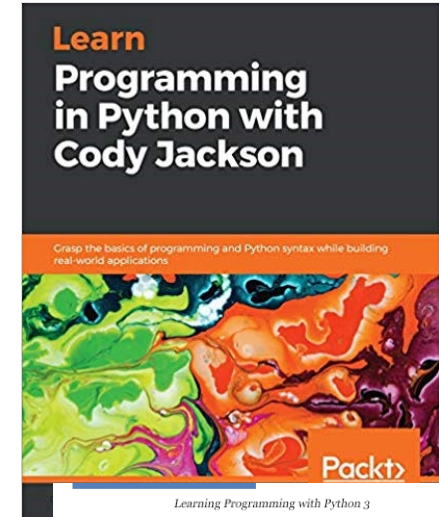


[1] <https://www.python.org/>

[2] <https://trends.google.com/trends/explore?date=2012-01-01%202023-01-01&q=Python%20data%20science,R%20data%20science,Python%20Machine%20Learning,R%20Machine%20Learning>

Resources

- “Learning to Program Using Python” by Cody Jackson.
- “The Coder’s Apprentice; Learning Programming with Python 3” by Pieter Spronck.
- <https://www.python.org/doc/>
- <https://www.learnpython.org/>
- <https://pythonprogramming.net/data-analysis-tutorials/>
- Deep Learning Specialization on Coursera.org, <https://www.coursera.org/specializations/deep-learning>
- Sample codes from Github.
- **Chollet, Francois. Deep learning with Python. Vol. 361. New York: Manning, 2018.**



FRANCOIS CHOLLET

MANNING
SHELTER ISLAND

Python IDEs and Editors (1 of 4)

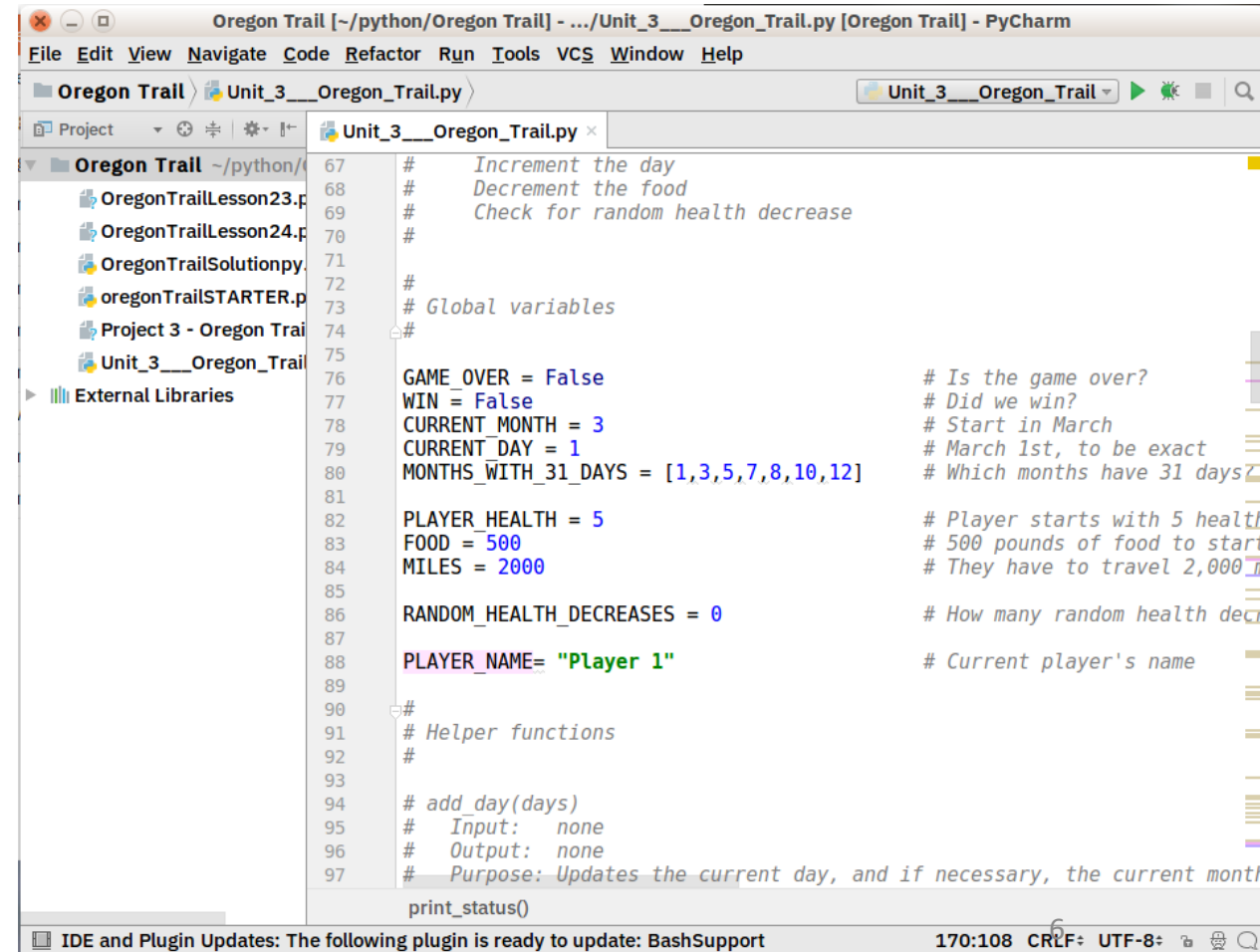
- An IDE (Integrated Development Environment)
 - A program dedicated to software development.
 - An editor designed to handle code (for example, syntax highlighting and auto-completion).
 - Build, execution, and debugging tools.
- There are many Python-specific IDEs:
 - PyCharm
 - Spyder
 - Google Colab



Python IDEs and Editors (2 of 4)



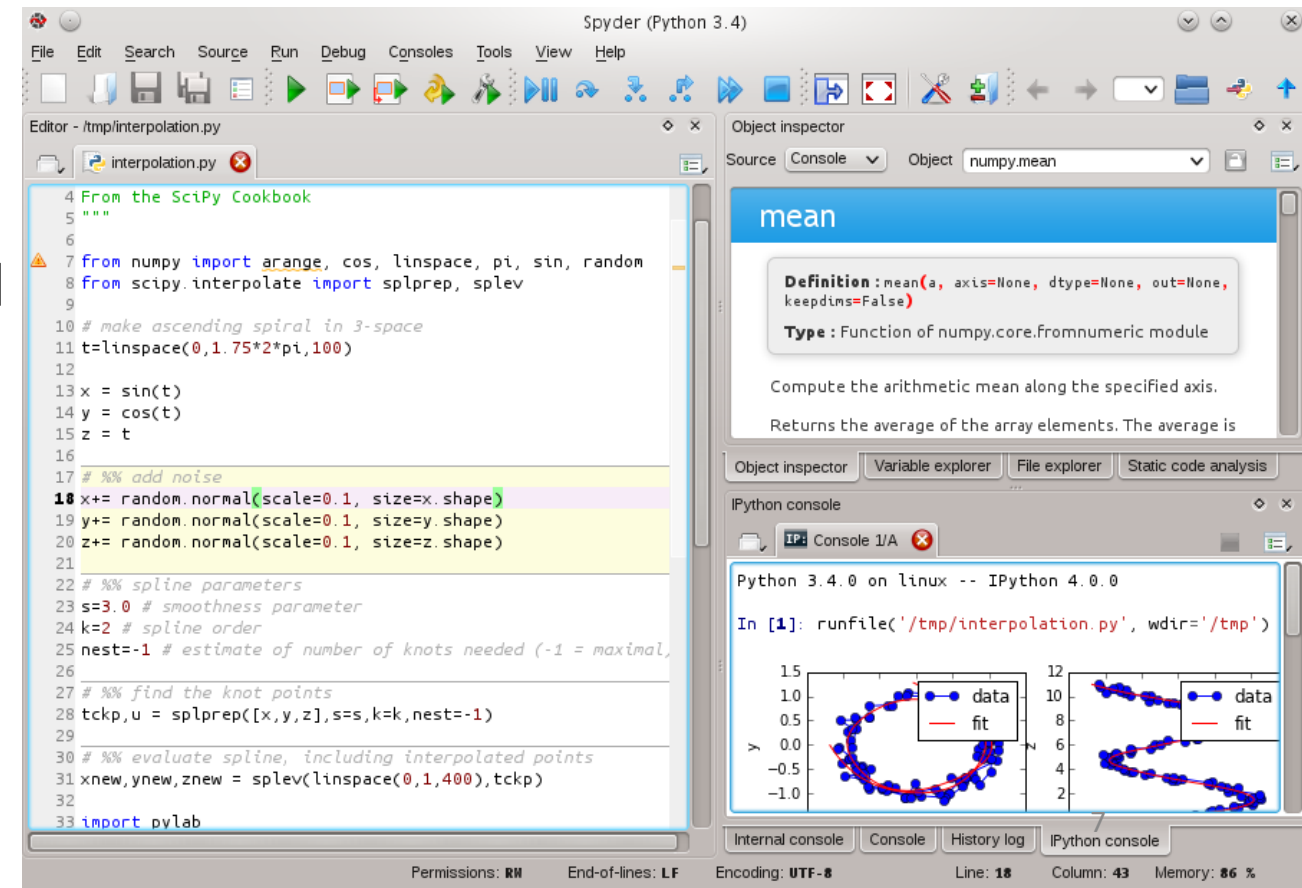
- PyCharm:
 - Full-featured, dedicated IDEs for Python.
 - You can create Python environments and install libraries easily or use available environments for multiple projects.
 - You can run and debug Python directly inside PyCharm.
 - You can access variables in the workspace during the debugging process.



Python IDEs and Editors (3 of 4)



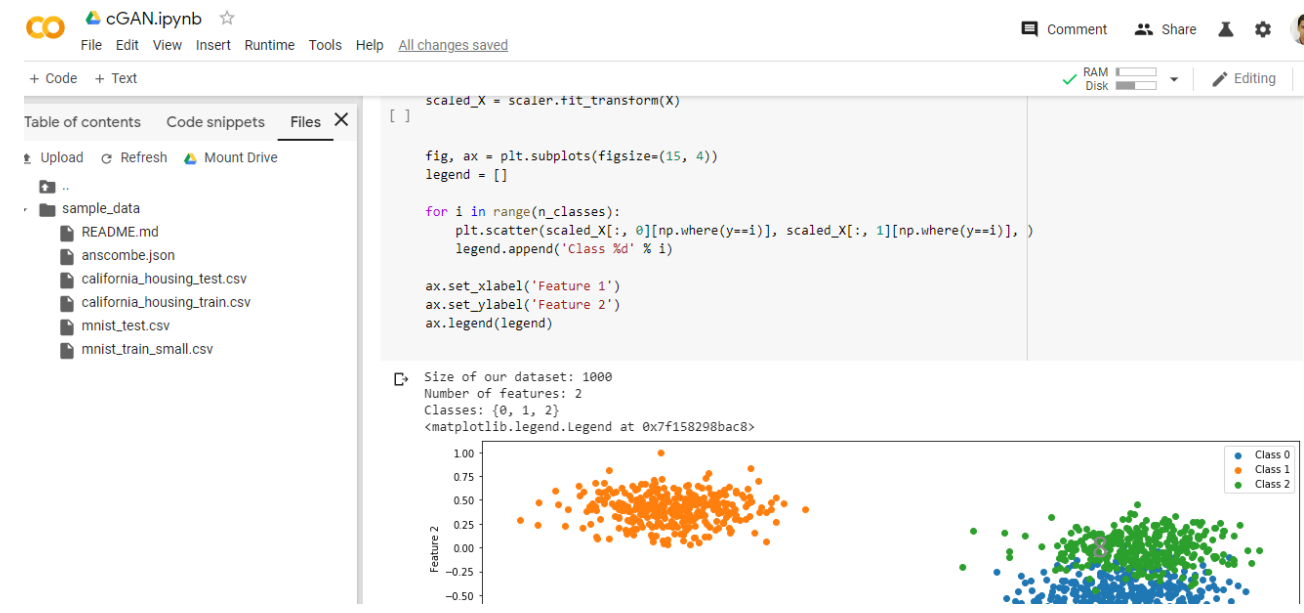
- Spyder:
 - Open-source Python IDE
 - Comes included with the Anaconda package manager distribution.
 - A code editor with robust syntax highlighting, Python code completion, and even an integrated documentation browser.
 - A special feature that is Spyder's "variable explorer".
 - The IPython/Jupyter notebook integration.



Python IDEs and Editors (4 of 4)



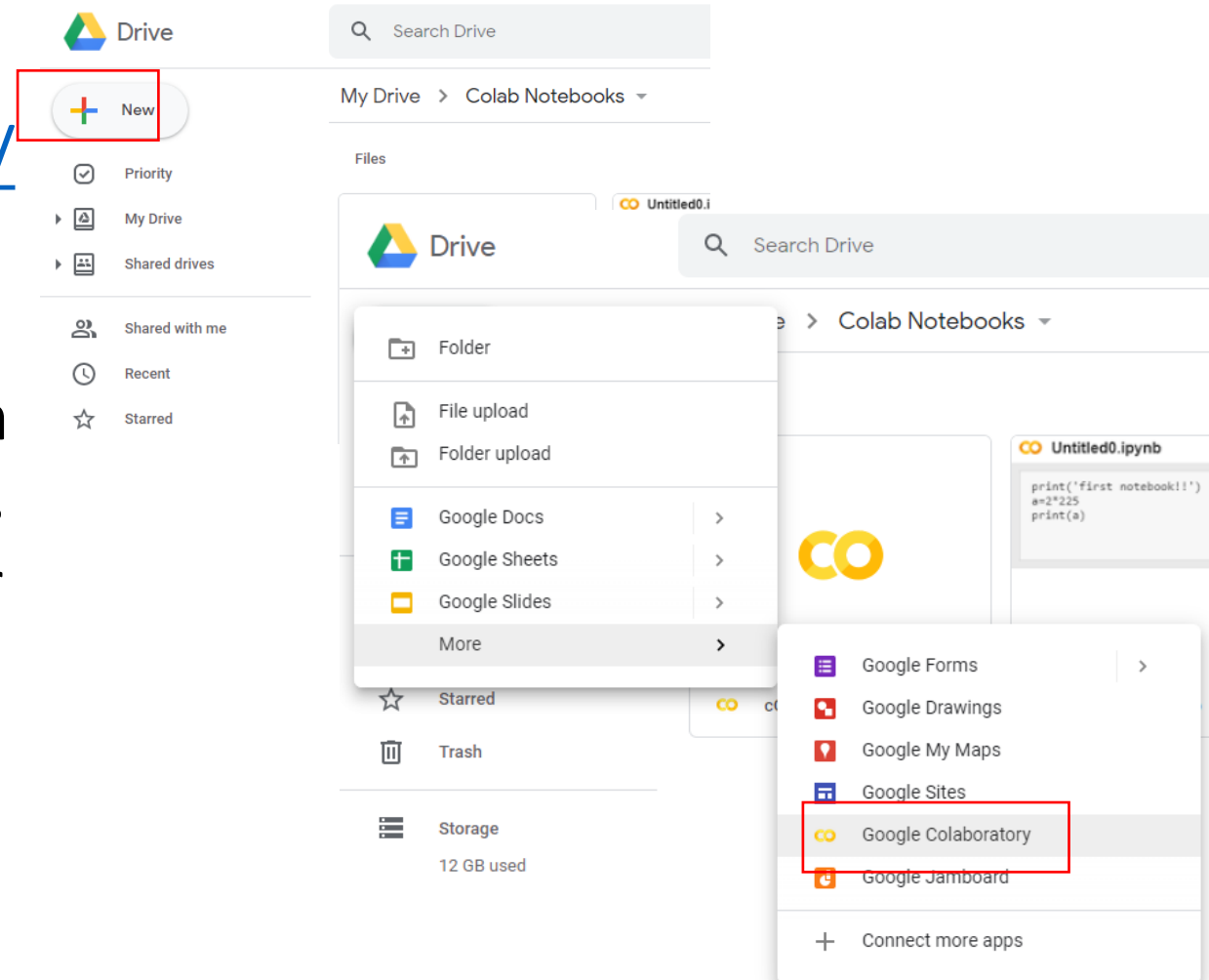
- Colab:
 - A free Jupyter notebook environment that runs entirely in the cloud.
 - Used to write and execute code in Python
 - The notebooks can be simultaneously edited by many people.
 - Colab supports many popular machine learning libraries.
 - Free Cloud service with free GPU.



Colab



- Go to <https://colab.research.google.com/> and then click on the NEW NOTEBOOK.
- Or Open Google Drive then click on New->More->Google Collaboratory.
 - Colab implicitly uses Google Drive for storing your notebooks.



Colab

- There are two terms:
- A kernel is a “computational engine” that executes the code contained in a notebook document.
- A cell is a container for or code to be executed by the kernel.

Change notebook name.

The screenshot displays the Google Colab web interface. At the top, the notebook is titled 'test2'. Below the title bar, there are tabs for '+ Code' and '+ Text'. The main area contains two code cells. The first cell contains a Python function definition and a call to the function, with its output 'Hello, Tim!' displayed below. The second cell contains a print statement and a calculation, with its output 'first notebook!!' and '450' displayed below. On the left side, there is a sidebar with a 'Table of contents' and a 'Code snippets' section. A red callout bubble points to the top left corner, indicating where to click to change the notebook name. Another red callout bubble points to the first code cell and its output, indicating that the code cell and its output are part of the notebook document. A third red callout bubble points to the second code cell, indicating that text can be added to describe the code.

File Edit View Insert Runtime Tools Help Last saved at 6:00 PM

+ Code + Text

Table of contents Code snippets Files

Filter code snippets

Rendering SymPy in Colab →

Saving data to Google Drive →

Saving data to Google Sheets →

Saving data with gsutil →

Saving data with the Cloud Storage Pyth... →

Serving resources →

Showing CV2 Images →

def say_hello(recipient):
 return 'Hello, {}'.format(recipient)
say_hello('Tim')

'Hello, Tim!'

[] print('first notebook!!')
a=2*225
print(a)

first notebook!!
450

Comment Share

RAM Disk

Editing

Add text cell

Colab

- Using text cell that support LaTeX to describe your code.

The screenshot displays the Google Colab interface. At the top, the menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. A red box highlights the '+ Text' button in the top-left corner. Below the menu, a sidebar on the left contains a 'Table of contents' and a 'Code snippets' section. The main workspace shows a code cell with the following Python code:

```
[2] print('first notebook!!')
a=2*225
print(a)
```

The output of the code cell is:

```
first notebook!!
450
Hello , world!
```

Below the code cell, a text cell is visible, containing LaTeX code for constraints and a trial vector calculation. The text cell is rendered with LaTeX formatting, showing constraints and a trial vector calculation. The constraints are:

- $3x_1 + 6x_2 + x_3 \leq 28$
- $7x_1 + 3x_2 + 2x_3 \leq 37$
- $4x_1 + 5x_2 + 2x_3 \leq 19$
- $x_1, x_2, x_3 \geq 0$

The trial vector is calculated as follows:

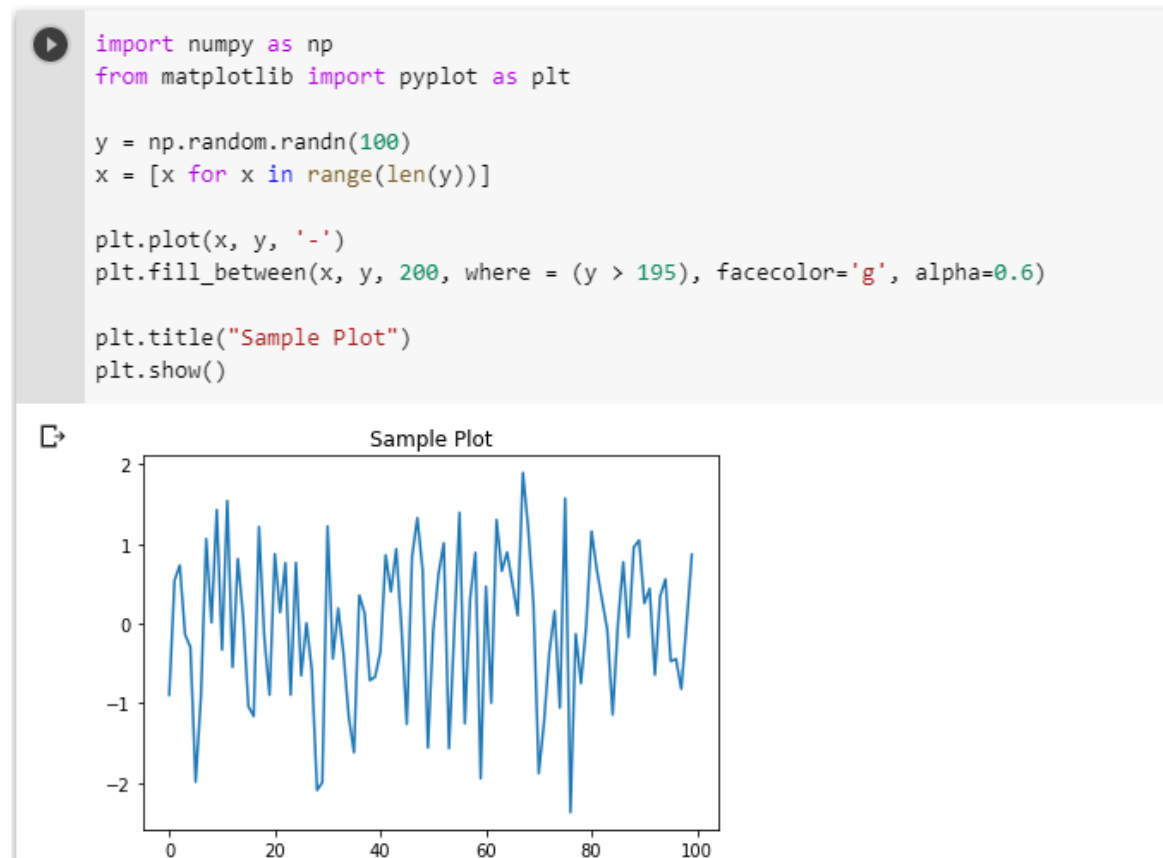
$$u_i(t) = x_i(t) + \beta(\hat{x}(t) - x_i(t)) + \beta \sum_{k=1}^{n_v} (x_{i1,k}(t) - x_{i2,k}(t))$$
$$f(x_1, x_2) = 20 + e - 20 \exp(-0.2 \sqrt{\frac{1}{n} (x_1^2 + x_2^2)}) - \exp(\frac{1}{n} (\cos(2\pi x_1) + \cos(2\pi x_2)))$$

The trial vector is calculated as follows:

$$x \in [-5, 5]$$
$$A_{m,n} = \begin{pmatrix} a_{1,1} > a_{1,2} > \dots > a_{1,n} \\ a_{2,1} > a_{2,2} > \dots > a_{2,n} \\ \vdots > \vdots > \ddots > \vdots \\ a_{m,1} > a_{m,2} > \dots > a_{m,n} \end{pmatrix}$$

Colab

- Colab supports rich outputs such as plots.



Expressions

- Python uses indentation for blocks, instead of curly braces.
 - Both tabs and spaces are supported, but the standard indentation requires standard Python code to use four spaces.

```
x = 1
```

```
if x == 1:
```

```
    # indented four spaces
```

```
    print("x is 1.")
```

- The print function

```
print( "Hello , world!" )
```

- You can display multiple things with one print

```
print( 'I', 'own', 'two', cars' )
```

```
print( 'I'+ ' own'+ ' two'+ ' cars' )
```



```
print('first notebook!!')
a=2*225
print(a)
print("I can't stand it")
print('I can\'t stand it')
print( 'I', 'own', 'two', 'apples')
print( 'I'+ ' own'+ ' two'+ ' apples')
```

```
first notebook!!
450
I can't stand it
I can't stand it
I own two apples
I  own two apples
```

Expressions – basic calculations

- Basic calculations

```
print( 15+4 )
print( 15-4 )
print( 15*4 )
print( 15/4 ) #division ->3.75
print( 15//4 ) #integer division or floor division-> 3
print( 15**4 ) #power print( 15%4 ) #modulo operator (%)
#takes the remainder of a division
```

```
19
11
60
3.75
3
50625
```

- Using parentheses to specify the calculations order.

```
print( (5*2) - (3+4)/2 )
print( ((5*2) - (3+4)) / 2 )
print( int(3.9) )
```

```
6.5
1.5
3
```

- Type casting

int() will return the value between the parentheses as an integer (rounding down if necessary)

float() will return the value between the parentheses as a float (adding .0 if necessary)

str() will return the value between the parentheses as a string

```
print("Float: " + str(3.5))
```

Expressions – data types

- Data types

- Strings

- Enclosed by either double quotes, or single quotes.
 - In principle, it does not matter which of the two you use, i.e., "orange" is equivalent to 'orange', but

```
print("I can't stand it")  
print('I can\'t stand it')
```

- Integers

- Floats

- Numbers with decimals.

```
A=3 #integer
```

```
A=3.0#float
```

```
print( (431 / 100) * 100 )
```

```
#answer 430.99999999999994 float,  
and not 431 integer
```

```
mystring = 'Hi'  
myfloat = 10.0  
myint = 20  
# testing code  
if mystring == 'Hi':  
    print("String: %s" % mystring)  
if isinstance(myfloat, float) and myfloat == 10.0:  
    print("Float: %.2f" % myfloat) #or print("Float: " + str(myfloat))  
if isinstance(myint, int) and myint == 20:  
    print("Integer: %d" % myint)
```

```
String: Hi  
Float: 10.00  
Integer: 20
```

Variables – how to create them?

- To create a variable (i.e., choose the variable name), you must “assign” it a value.

```
x=5 # you can also use functions such as int()...
```

```
y = 3.0 #type(y)
```

```
print( "x * y =", x * y )
```

- you do not “declare” the type of a variable, but a variable still has a type
- Variable names
 - A variable name must consist of only letters, digits, and/or underscores (_)
 - A variable name must start with a letter or an underscore
 - A variable name should not be a reserved word
 - Variable names are case sensitive

Variables - conventions

- Never choose variable names that are also the names of functions
- Choose variable names that are in some way meaningful to the code
- Tend to use only lower-case letters in variable names with underscores.

```
▶ pi = 3.14159265  
  radius = 7.5  
  height = 8.25  
  volume_of_cone = pi * radius * radius * height / 3  
  print( 'The volume of the cone =%.2f'% volume_of_cone )
```

The volume of the cone =485.97

- Comments: by using hash mark (#)

Simple Functions

- Type casting
 - float(), int(), str()
- Calculations

```
x = -2
y = 3
z = 1.27
print( abs( x ) )
print( max( x, y, z ) )
print( min( x, y, z ) )
print( pow( x, y ) )
print( round( z, 1 ) ) #round the first parameter to the number of decimals
                        #specified by the second parameter
```

```
2
3
-2
-8
1.3
```

- len() returns the length of the first parameter.
`print( len( ' can ' ) )` #->5 counting the spaces
- Input() asks the user to supply a string value.

```
number = input( "Please enter a number: " )
number = float( number )
print( "Your number squared is", number * number )
```

```
Please enter a number: 3.6
Your number squared is 12.96
```

- format() allows creating a formatted string.

```
print( "{:.4f}".format( 7/11 ) )
print( "Float: %.2f" % (7/11) )
print( "The first 3 numbers are {}, {} and {:.3f}.".format("one", 2, 3.3332 ) )
```

```
0.6364
Float: 0.64
The first 3 numbers are one, 2 and 3.333.
```

Simple Functions - modules

- Modules:
 - Python offers a large assortment of so-called “modules,” which contain many more useful functions.
 - You have to import the module, by `import <module n>`
 - You can import only specific functions from a module, by stating:
`from <module n> import <function1>, <function2>, <function3>`
- math module contains some useful mathematical functions:
 - `exp()`, `log()`, `log10()`, `sqrt()`, ...
`from math import sqrt`
`print( sqrt( 4 ) )`
- random module contains functions that return pseudo-random numbers:
 - `random()` returns a random float in the range [0, 1)
 - `randint()` gets two parameters. It returns a random integer in the range for which the two parameters are boundaries.
 - `seed()` initializes the random number generator of Python.

```
from random import randint, seed
seed()
print( "A random number between 1 and 10 is", randint( 1, 10 ) )
```

A random number between 1 and 10 is 10

Conditions

- Comparisons:
 - < less than
 - <= less than or equal to
 - == equal to
 - >= equal to or greater than
 - > greater than
 - != not equal
- The `in` operator tests if the value to the left side of the operator is found in a “collection” to the right side of the operator.

```
print( "x" in "Python" )  
print( "p" in "Python" )
```

False
False

- Logical operators:
 - `and`, `or`, and `not`.

- Conditional statements:

```
x = 4  
if x > 2:  
    print( "x is bigger than 2" )  
elif x==2:  
    print( "x equals to 2" )  
else:  
    print( "x is smaller than 2" )
```

x is bigger than 2

Iterations – while and for

- while <Boolean expression>:

<statements>

else:

<statements>

```
▶ total = 0
count = 0
print('Please enter 5 integer numbers:')
while count < 5:
    total += int(input( "number "+str(count+1)+": " ))
    count += 1
print( "Total is", total )
```

```
Please enter 5 integer numbers:
number 1: 2
number 2: 3
number 3: 96
number 4: 6
number 5: 6
Total is 113
```

- for <variable> in <collection>:

<statements>

```
[ ] fruit = "banana"
    for letter in fruit:
        print( letter )

    for x in range(5):
        print( x )
```

```
b
a
n
a
n
a
0
1
2
3
4
```

Iterations - break

- `break` allows you to break out of a loop prematurely.
#Looking for a number that starts with a 1, and when you transfer that 1 to the end of the number, the result is a number that is three times as high.

```
i = 1
while i <= 10000000:
    num1 = int( "1" + str( i ) )
    num2 = int( str( i ) + "1" )
    if num2 == 3 * num1:
        break
    i += 1
print( num2 , "is three times", num1 )
```

428571 is three times 142857

#checking a list of grades for a student. If all grades are 5.5 or higher, the student passes.

```
grades=[8, 7.5, 9, 6, 6, 6, 5.5, 7, 5.5, 8, 7, 7.5]
for grade in grades:
    if grade < 5.5:
        print( "The student fails!" )
        break
    else:
        print( "The student passes!" )
```

☞ The student passes!

Iterations - continue

- When the `continue` statement is encountered in the code block of a loop, the current cycle ends immediately and the code loops back to the start of the loop.

#Printing all numbers between 1 and 100 that cannot be divided by 2 or 3,
do not end in a 7 or 9, and do not consist of two equal digits

```
for num in range( 1, 101 ):
    if num%2 == 0:
        continue
    if num%3 == 0:
        continue
    if num%10 == 7:
        continue
    if num%10 == 9:
        continue
    if num > 9:
        if num%10 == int( num / 10 ): #excluding 11, 55, ...
            continue
    print( num )
```

1
5
13
23
25

- Nested loops

```
[ ] for i in range( 4 ):
    for j in range( i+1, 4 ):
        print( "({}-{})".format( i, j ) )
```

(0-1)
(0-2)
(0-3)
(1-2)
(1-3)
(2-3)

Example 1

- Ask the user for two numbers between 0 and 1000 in a loop. For every two numbers that the user enters, show their multiplication.
- Allow the user to stop the program when s/he enters zero for any of the numbers. For some reason, if the numbers are dividers of each other, that is an error and the program also stops, but with an error message.

```
from sys import exit
while True:
    x = int(input( "Enter number 1: " ))
    if x == 0:
        break
    y = int(input( "Enter number 2: " ))
    if y == 0:
        break
    if (x < 0 or x > 1000) or (y < 0 or y > 1000):
        print( "The numbers should be between 0 and 1000" )
        continue
    if x%y == 0 or y%x == 0:
        print( "Error: the numbers cannot be dividers" )
        exit()
    print( "Multiplication of", x, "and", y, "gives", x * y )
print( "Goodbye!" )
```

```
Enter number 1: 5
Enter number 2: 22
Multiplication of 5 and 22 gives 110
Enter number 1: -2
```

Example 2

- If you roll five 6-sided dice, how big is the probability that you roll five sixes?
- Imitate the rolling of a die using `randint()`.
- Check whether they all show a 6.
- Repeat this for many times

```
▶ from random import randint
tests=1000000
success=0
for testi in range (tests):
    for d in range (5):
        if randint(1,6)!=6:
            break
    else: #The else clause executes after the loop completes normally.
        #This means that the loop did not encounter a break statement.
        success+=1
print( "Chance at five sixes is", success / TESTS )
print("The optimal answer is ", 1/(6**5))
```

```
Chance at five sixes is 0.00013
The optimal answer is  0.0001286008230452675
```

- The answer is $1/(6^5)$